

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Poster Presentation

Course Code:	DSA0405	Course Title:	FUNDAMENTALS OF DATA SCIENCE
CO Assessed:	CO3 –Poster Presentation	Assessment:	Assessment Tool 1– Poster Presentation
Weightage:	40% of CIA	Total Marks:	100
CO3 Statement:	Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BL4)		
Student Name:	B Tharun Kumar	Reg. No.:	192424356
Section / Batch:	AI-DS	Date:	27/08/26

Code of Conduct

I B Tharun Kumar – 192424356 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *B Tharun Kumar*

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	

Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

DSA0404 – Fundamentals of Data Science
CO5-AT2 Coding Challenge

Q1. The following Python program implements a CART decision tree and creates a graphical visualization of the tree from scratch using Matplotlib. It displays the feature splits, thresholds, branch conditions, predicted classes, and class distributions at leaf nodes. It does not use `sklearn.tree.plot_tree()` or `Graphviz`. The graphical visualizer is based on recursively calculating node positions and drawing the nodes and connecting edges.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
```

```
# =====
# 1. TREE NODE
# =====
```

```
class TreeNode:
```

```
    def __init__(
        self,
        feature=None,
        threshold=None,
        left=None,
        right=None,
        class_counts=None,
        predicted_class=None
    ):
        self.feature = feature
        self.threshold = threshold
        self.left = left
        self.right = right
        self.class_counts = class_counts
        self.predicted_class = predicted_class
```

```
# =====
# 2. CART DECISION TREE
# =====
```

```
class CARTDecisionTree:
```

```
def __init__(self, max_depth=3, min_samples_split=2):
```

```
    self.max_depth = max_depth
    self.min_samples_split = min_samples_split
```

```
    self.root = None
    self.feature_names = None
    self.class_names = None
```

```
# -----
# Calculate Gini impurity
# -----
```

```
def gini(self, y):
```

```
    if len(y) == 0:
        return 0
```

```
    counts = Counter(y)
```

```
    impurity = 1.0
```

```
    for count in counts.values():
```

```
        probability = count / len(y)
```

```
        impurity -= probability ** 2
```

```
    return impurity
```

```
# -----
# Find the best split
# -----
```

```
def best_split(self, X, y):
```

```
    n_samples, n_features = X.shape
```

```
    if n_samples <= 1:
        return None, None
```

```
    parent_gini = self.gini(y)
```

```
    best_gain = 0
```

```
    best_feature = None
```

```

best_threshold = None

for feature in range(n_features):

    values = np.unique(X[:, feature])

    if len(values) <= 1:
        continue

    # Candidate thresholds
    thresholds = (
        values[:-1] + values[1:]
    ) / 2

    for threshold in thresholds:

        left_mask = X[:, feature] <= threshold
        right_mask = X[:, feature] > threshold

        y_left = y[left_mask]
        y_right = y[right_mask]

        if len(y_left) == 0 or len(y_right) == 0:
            continue

        left_weight = len(y_left) / n_samples
        right_weight = len(y_right) / n_samples

        weighted_gini = (
            left_weight * self.gini(y_left)
            + right_weight * self.gini(y_right)
        )

        # Gini gain
        gain = parent_gini - weighted_gini

        if gain > best_gain:

            best_gain = gain
            best_feature = feature
            best_threshold = threshold

    return best_feature, best_threshold

# -----
# Build the tree recursively

```

```
# -----
```

```
def build_tree(self, X, y, depth=0):
```

```
    # Count samples belonging to each class
```

```
    class_counts = Counter(y)
```

```
    # Find majority class
```

```
    predicted_class = max(
```

```
        class_counts,
```

```
        key=class_counts.get
```

```
    )
```

```
    # Stopping conditions
```

```
    if (
```

```
        depth >= self.max_depth
```

```
        or len(y) < self.min_samples_split
```

```
        or len(class_counts) == 1
```

```
    ):
```

```
        return TreeNode(
```

```
            class_counts=class_counts,
```

```
            predicted_class=predicted_class
```

```
        )
```

```
    # Find the best split
```

```
    feature, threshold = self.best_split(X, y)
```

```
    # If no split is possible, create a leaf
```

```
    if feature is None:
```

```
        return TreeNode(
```

```
            class_counts=class_counts,
```

```
            predicted_class=predicted_class
```

```
        )
```

```
    # Split the data
```

```
    left_mask = X[:, feature] <= threshold
```

```
    right_mask = X[:, feature] > threshold
```

```
    X_left = X[left_mask]
```

```
    y_left = y[left_mask]
```

```
    X_right = X[right_mask]
```

```
    y_right = y[right_mask]
```

```

# Recursively create child nodes
left_child = self.build_tree(
    X_left,
    y_left,
    depth + 1
)

right_child = self.build_tree(
    X_right,
    y_right,
    depth + 1
)

return TreeNode(
    feature=feature,
    threshold=threshold,
    left=left_child,
    right=right_child,
    class_counts=class_counts,
    predicted_class=predicted_class
)

# -----
# Train the CART tree
# -----

def fit(
    self,
    X,
    y,
    feature_names=None,
    class_names=None
):
    X = np.asarray(X)
    y = np.asarray(y)

    self.feature_names = feature_names

    if class_names is None:
        self.class_names = [
            str(c) for c in np.unique(y)
        ]

    else:

```

```

        self.class_names = class_names

    self.root = self.build_tree(X, y)

    return self

# -----
# Predict one sample
# -----

def predict_one(self, x, node):

    # If leaf node, return prediction
    if node.feature is None:

        return node.predicted_class

    # Go to left child
    if x[node.feature] <= node.threshold:

        return self.predict_one(
            x,
            node.left
        )

    # Go to right child
    return self.predict_one(
        x,
        node.right
    )

# -----
# Predict multiple samples
# -----

def predict(self, X):

    X = np.asarray(X)

    predictions = []

    for x in X:

        predictions.append(
            self.predict_one(

```

```

        x,
        self.root
    )
)

```

```

return np.array(predictions)

```

```

# =====
# 3. GRAPHICAL TREE VISUALIZER
# =====

```

```

class TreeVisualizer:

```

```

    def __init__(self, tree):

```

```

        self.tree = tree

```

```

        self.feature_names = tree.feature_names
        self.class_names = tree.class_names

```

```

        self.positions = {}

```

```

# -----
# Count leaf nodes
# -----

```

```

    def count_leaves(self, node):

```

```

        if node is None:
            return 0

```

```

        if node.feature is None:
            return 1

```

```

        return (
            self.count_leaves(node.left)
            + self.count_leaves(node.right)
        )

```

```

# -----
# Calculate positions of nodes
# -----

```

```

    def calculate_positions(
        self,

```

```

node,
depth=0,
x_min=0,
x_max=1
):

if node is None:
    return

# Position of current node
x = (x_min + x_max) / 2

y = -depth

self.positions[id(node)] = (x, y)

# Position of left child
if node.left is not None:

    self.calculate_positions(
        node.left,
        depth + 1,
        x_min,
        x
    )

# Position of right child
if node.right is not None:

    self.calculate_positions(
        node.right,
        depth + 1,
        x,
        x_max
    )

# -----
# Create node label
# -----

def get_label(self, node):

    # Leaf node
    if node.feature is None:

        distribution = []

```

```

for class_id, count in sorted(
    node.class_counts.items()
):

    class_name = self.class_names[class_id]

    distribution.append(
        f'{class_name}: {count}'
    )

predicted_name = (
    self.class_names[node.predicted_class]
)

return (
    f'LEAF\n'
    f'Class: {predicted_name}\n'
    f'Samples: '
    f'{sum(node.class_counts.values())}\n'
    f'Distribution:\n'
    f'{', '.join(distribution)}'
)

# Internal node
feature = node.feature

if self.feature_names is not None:

    feature_name = (
        self.feature_names[feature]
    )

else:

    feature_name = (
        f'Feature {feature}'
    )

return (
    f'{feature_name}\n'
    f'<= {node.threshold:.2f}\n'
    f'Samples: '
    f'{sum(node.class_counts.values())}'
)

```

```
# -----  
# Draw nodes and edges  
# -----
```

```
def draw_node(self, ax, node):
```

```
    if node is None:  
        return
```

```
    x, y = self.positions[id(node)]
```

```
# -----  
# Draw edge to left child  
# -----
```

```
if node.left is not None:
```

```
    child_x, child_y = (  
        self.positions[id(node.left)]  
    )
```

```
    ax.plot(  
        [x, child_x],  
        [y, child_y],  
        linewidth=1.5  
    )
```

```
    # Branch label  
    ax.text(  
        (x + child_x) / 2,  
        (y + child_y) / 2 + 0.05,  
        "True",  
        fontsize=9,  
        ha="center"  
    )
```

```
# -----  
# Draw edge to right child  
# -----
```

```
if node.right is not None:
```

```
    child_x, child_y = (  
        self.positions[id(node.right)]  
    )
```

```

ax.plot(
    [x, child_x],
    [y, child_y],
    linewidth=1.5
)

# Branch label
ax.text(
    (x + child_x) / 2,
    (y + child_y) / 2 + 0.05,
    "False",
    fontsize=9,
    ha="center"
)

# -----
# Select node appearance
# -----

if node.feature is None:

    box_color = "lightgreen"

else:

    box_color = "lightblue"

# -----
# Draw node
# -----

ax.text(
    x,
    y,
    self.get_label(node),
    ha="center",
    va="center",
    fontsize=9,
    bbox=dict(
        boxstyle="round,pad=0.6",
        facecolor=box_color,
        edgecolor="black",
        linewidth=1.5
    )
)

```

```

# Draw children
self.draw_node(
    ax,
    node.left
)

self.draw_node(
    ax,
    node.right
)

# -----
# Display complete tree
# -----

def plot(self):

    # Calculate node positions
    self.calculate_positions(
        self.tree.root
    )

    # Find minimum depth
    depths = [
        y
        for x, y in self.positions.values()
    ]

    min_depth = min(depths)

    # Calculate figure size
    number_of_leaves = self.count_leaves(
        self.tree.root
    )

    width = max(
        12,
        number_of_leaves * 3
    )

    height = max(
        6,
        abs(min_depth) * 2.5 + 3
    )

    # Create figure

```

```
fig, ax = plt.subplots(  
    figsize=(width, height)  
)
```

```
# Draw tree  
self.draw_node(  
    ax,  
    self.tree.root  
)
```

```
# Set plot limits  
ax.set_xlim(  
    -0.05,  
    1.05  
)
```

```
ax.set_ylim(  
    min_depth - 1,  
    1  
)
```

```
# Remove axes  
ax.axis("off")
```

```
# Title  
plt.title(  
    "CART Decision Tree",  
    fontsize=18,  
    fontweight="bold"  
)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# =====  
# 4. SAMPLE DATASET  
# =====
```

```
# Features:  
# Age  
# Income  
# Credit Score
```

```
X = np.array([
```

```
[22, 25000, 580],
[25, 30000, 600],
[28, 32000, 610],
[30, 35000, 620],
[35, 40000, 640],
[40, 45000, 660],
[42, 50000, 680],
[45, 55000, 700],
[48, 60000, 720],
[50, 65000, 740],
[55, 70000, 760],
[60, 80000, 780]
], dtype=float)
```

```
# Target classes:
# 0 = No
# 1 = Yes
```

```
y = np.array([
    0,
    0,
    0,
    0,
    0,
    0,
    1,
    1,
    1,
    1,
    1,
    1,
    1
])
```

```
# =====
# 5. FEATURE AND CLASS NAMES
# =====
```

```
feature_names = [
    "Age",
    "Income",
    "CreditScore"
]
```

```
class_names = [
```

```
"No",  
"Yes"  
]
```

```
# =====  
# 6. TRAIN CART  
# =====
```

```
tree = CARTDecisionTree(  
    max_depth=3,  
    min_samples_split=2  
)
```

```
tree.fit(  
    X,  
    y,  
    feature_names=feature_names,  
    class_names=class_names  
)
```

```
# =====  
# 7. DISPLAY GRAPHICAL TREE  
# =====
```

```
visualizer = TreeVisualizer(tree)
```

```
visualizer.plot()
```

```
# =====  
# 8. TEST THE MODEL  
# =====
```

```
X_test = np.array([  
    [27, 30000, 600],  
    [45, 55000, 700],  
    [58, 75000, 770]  
, dtype=float)
```

```
predictions = tree.predict(X_test)
```

```
print("\nPredictions:")
```

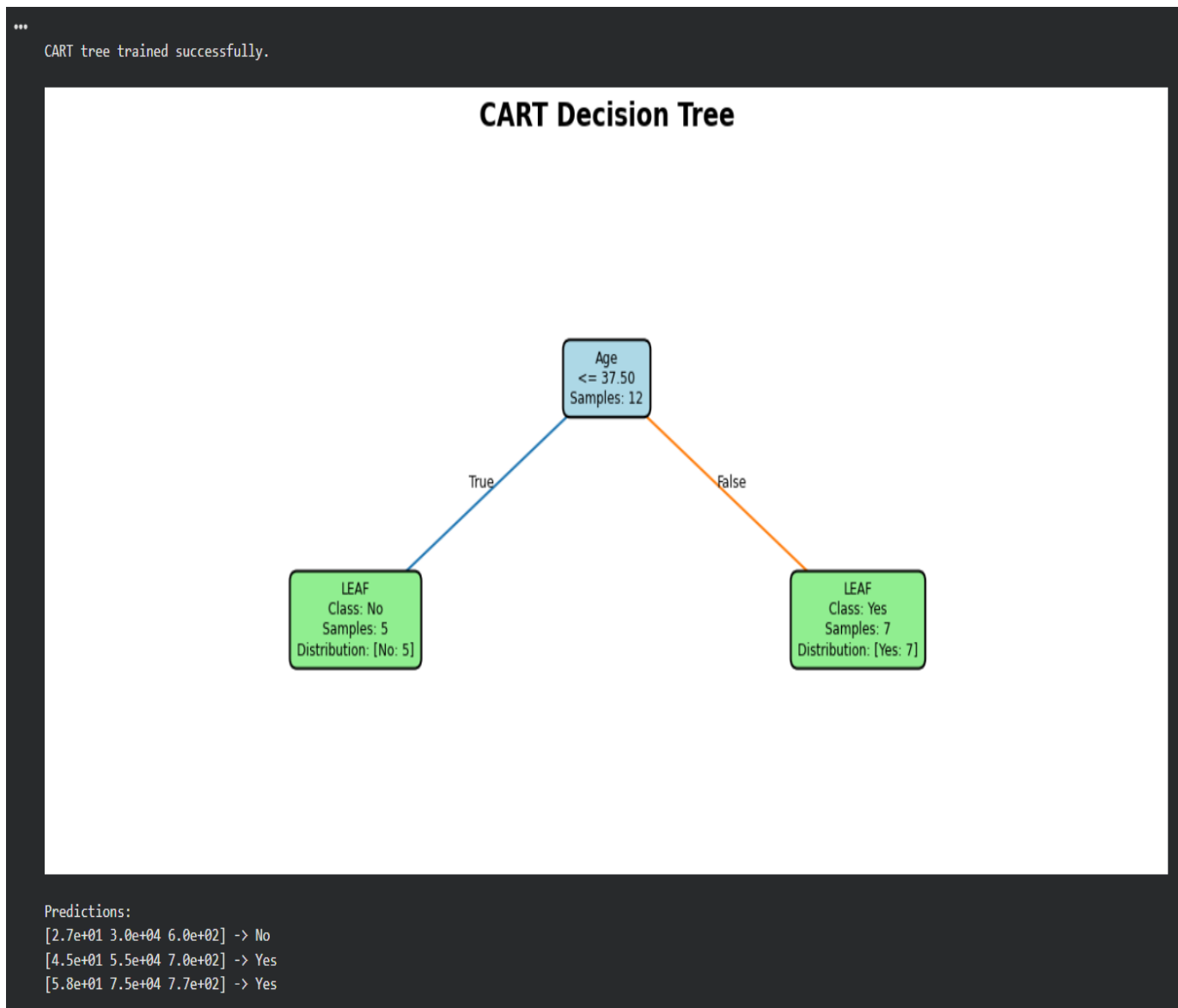
```

for sample, prediction in zip(
    X_test,
    predictions
):

    print(
        f"{sample} -> "
        f"{class_names[prediction]}"
    )

```

Output:



Geotag:



